

Comprehensive Research Report: Drone SLAM, Obstacle Avoidance, and Cloud Simulation

Author: Soham

Assignment: BeRAM AI/ML Research Assignment

Date: 7th October 2026

Video:

https://drive.google.com/drive/folders/1TTvHIMl8aO4yN5_ErkoZsdJs5aWz24zd?usp=s_haring

Scope: Simulation-only implementation using WSL2 (Ubuntu 22.04 LTS), ROS 2 Humble, Gazebo Harmonic, ArduPilot SITL, and Cartographer SLAM. No physical hardware or paid cloud resources were consumed.

1. How I Approached This Assignment

- **The False Start (VirtualBox Trap):** I initially thought of using a traditional VirtualBox Ubuntu VM despite other options, which cost me 2 full days of frustration due to severe USB pass-through, hardware acceleration, and virtualization bottlenecks.
- **The Pivot to WSL2:** Realizing VirtualBox was a dead end for real-time physics simulation, I wiped the slate clean and switched onto WSL2 (Ubuntu 22.04 LTS), which immediately unlocked better hardware integration and stable terminal workflows.
- **Level-by-Level Escalation:** From that turning point, I structured my execution systematically—first conquering basic SITL flight setup (Level 1), then building the 3D maze environment and LiDAR mapping pipeline (Level 2), and finally exploring autonomous navigation and cloud scalability (Level 3).

2. System Architecture & Environment Setup

2.1 Software Stack Overview

The operational environment relied on a precise configuration of modern open-source robotics frameworks:

- **Operating System:** Ubuntu 22.04 LTS running via Windows Subsystem for Linux (WSL2).
- **Simulation Engine:** Gazebo Harmonic (gz-sim) utilizing the OGRE rendering engine for visual display.
- **Middleware & Communication:** ROS 2 Humble Hawksbill, leveraging ROS-Gazebo bridges (ros_gz_bridge) to translate simulation topics into ROS message types.
- **Flight Control:** ArduPilot SITL (arduropter) paired with MAVProxy telemetry and ground control interface.
- **Mapping Framework:** Cartographer SLAM configured for 2D occupancy grid generation.

3. Simultaneous Localization and Mapping (SLAM)

3.1 The Problem, and Why GPS Alone Is Not Enough

SLAM (Simultaneous Localization and Mapping) means building a map of an unknown place while also working out where the robot is inside that map. The two jobs depend on each other: a good map needs a good position, and a good position needs a good map. GPS alone is not enough for several reasons. It does not work inside buildings, tunnels or caves, and it is weak between tall buildings because signals bounce or are blocked. Its accuracy is usually metres, which is too coarse for flying through a corridor. Most importantly, GPS only tells the drone where it is on Earth; it says nothing about the walls and obstacles around it. A maze like mine would be invisible to GPS even if the signal were perfect. In my simulation the drone still had simulated GPS for flying, so I did not achieve true GPS-denied flight; the map came purely from the lidar.

3.2 Main Types of SLAM

- **Lidar:** Match laser scans. *Good:* Accurate, dark-proof. *Weak:* Weight, power, cost. *Example:* Cartographer, GMapping.
- **Visual:** Track image features. *Good:* Light, cheap, rich. *Weak:* Lighting, scale, texture. *Example:* ORB-SLAM2.
- **Filter-based:** Recursive estimate. *Good:* Low compute. *Weak:* Drift is hard to fix. *Example:* EKF-SLAM, GMapping.
- **Graph-based:** Optimise pose graph. *Good:* Fixes drift on loop closure. *Weak:* More compute and memory. *Example:* Cartographer, ORB-SLAM2.

3.3 The Package I Used: Cartographer

Cartographer is a lidar-based, graph-based 2D SLAM system. In simple terms it works in two layers. First, local SLAM: each new scan is aligned (scan-matched) with the most recent small piece of map, called a submap, which gives the drone's pose and adds the scan to that submap. In my configuration this uses Ceres scan matching with online correlative matching switched on, and it also uses the odometry from Gazebo. Second, global SLAM: when a submap is finished, Cartographer looks for matches between new scans and older submaps; each match is a constraint (a loop closure), and a pose graph optimization runs every 30 nodes in my configuration to correct accumulated drift. A separate occupancy-grid node turns the submaps into the /map topic that RViz shows and that map_saver_cli saves.

Getting it to work needed three fixes that taught me how it fits together: the odometry topic had to be named /odometry, the frames imu_link and base_scan had to be linked to base_link with static transforms (I used zero offsets, which is an approximation), and the clock had to be the simulation clock. Limits I noticed: a low 5 Hz scan rate means the drone must fly slowly, and the map is only correct if the drone stays level and at one height, because the lidar sees a single flat slice.

4. Obstacle Avoidance

4.1 Global Path Planning Versus Local Avoidance

Global path planning uses a map of the whole environment to compute a complete route from start to goal before moving, for example with Dijkstra's algorithm or A^* . It can find the best route and knows about dead ends, but it needs an accurate map and has to re-plan when something unexpected appears. Local avoidance looks only at what the sensors see right now and reacts within a short distance, for example the dynamic window approach or the vector field histogram. It is fast and handles surprises, but it has no memory of the layout, so it can get stuck in a dead end. Real robots usually combine both: a global plan for the route and a local avoidance for safety. ArduPilot's built-in options follow the same split: Simple Avoidance and BendyRuler are local, while the Dijkstra option plans around fences and known obstacles.

4.2 The Method I Used, and Why

I used my own lidar-based local avoider. ArduPilot's built-in avoidance needs the lidar fed into the autopilot as a proximity sensor, which my ROS-based setup did not provide, and a ROS 2 planner such as Nav2 needs much more configuration than the time allowed. Writing my own logic needed only the /scan topic I already had, and I could explain every line. The target is a point 6 m ahead of the drone's starting heading. On every scan the node does the following:

1. reads the drone's position and heading from ArduPilot over MAVLink and computes the direction of the target;
2. tests directions from -90 to +90 degrees in 15-degree steps; a direction is free if nothing in a window around it is closer than a safety distance SAFE;
3. chooses the free direction closest to the target direction;
4. sends a body-frame velocity command in that direction with zero vertical speed, so height stays constant and the 2D slice stays consistent;
5. stops when it is within 0.5 m of the target.

My first attempt used $\text{SAFE} = 1.8 \text{ m}$, a window of 20 degrees either side and $0.6 \sim \text{m/s}$.

The drone hit the box and fell. I reasoned that a $0.5 \times 1.5 \text{ m}$ box can partly sit outside a narrow window, and that at 5 Hz the drone travels too far between scans, so I raised SAFE to 2.5 m, widened the window to 30 degrees, lowered the speed to $0.4 \sim \text{m/s}$ and moved the box further away to give room to steer. The final behaviour is shown in the demo video. This is a reactive method only: it can get trapped in a dead end, it cannot see anything above or below the lidar slice, and it does not use the SLAM map at all.

5. From Simulation to a Real Drone

- **Sensor choice and weight:** In simulation the lidar weighs nothing and draws no power. A real spinning 2D lidar adds payload (typically hundreds of grams, so I would check the datasheet), uses battery power, and cuts flight time. It must be mounted clear of the propellers and the frame so that it does not scan its own drone. Lighter options, such as small time-of-flight rangefinders or a depth camera, trade coverage for weight.
- **Onboard computer:** My laptop ran Gazebo, SITL, ROS 2 and Cartographer together and still managed only 5 Hz. A real drone needs a small companion computer (a Raspberry Pi or Jetson class board) running ROS 2 and the SLAM node, connected to the flight controller by MAVLink. Its CPU, heat, weight and power budget limit the scan rate and the map size, so SLAM parameters would need tuning.
- **Vibration:** Motors and propellers shake the airframe. This adds noise to the IMU, blurs the lidar scans, and can loosen mounts, none of which exists in the simulator. Real drones use soft mounts or dampers for the flight controller, balanced propellers and filtering.
- **Limits of a 2D lidar on a vehicle that tilts and changes height:** A quadcopter has to tilt to move, and when it tilts the scan plane tilts with it. Range readings are then no longer measured along a horizontal slice: walls look too far or too close, and the lidar may hit the floor or ceiling. Cartographers can use IMU data to compensate for this, but my configuration had it switched off. When the drone changes height, the slice hits different parts of the scene (a table top, a door frame), so scans taken at different heights no longer agree and the map gets corrupted. A 2D lidar also cannot see obstacles above or below its plane, such as hanging cables or table edges, and it is poor on glass and shiny surfaces. For real indoor flight I would add a downward rangefinder for height, use the IMU to level the scans, and consider a 3D lidar or depth camera. For GPS-denied flight the SLAM pose would have to be fed into the autopilot's EKF as an external position source, which I did not do.

6. Deployment on AWS (Research Only)

This section is desk research. I did not create any AWS resource. I checked the status of each service on 7 October 2026 using AWS documentation and pages; the prices come from sites that republish AWS list prices, so they should be re-checked in the official AWS Pricing Calculator before anyone relies on them.

6.1 Options That Exist Today

- **AWS RoboMaker (managed ROS/Gazebo simulation):** Discontinued. Support ended on 10 September 2025 and the console and resources are no longer accessible. AWS's sample Gazebo world repositories were archived on 21 July 2026. *Fit:* Not an option. Any guide that starts with RoboMaker is out of date.
- **AWS Batch (containers):** Available; AWS's own notice points RoboMaker users here. AWS says it is best for headless batch simulation at scale, not interactive GUI use. *Fit:* Good for running many scenarios without a window (for example my parameter experiments), with results saved to S3.

- **EC2 GPU instance + Amazon DCV:** Available. NICE DCV is now called Amazon DCV; there is no extra charge for DCV on EC2, and it has a browser client on port 8443. *Fit:* Best fit. One interactive Gazebo + RViz desktop, viewed in a browser.
- **Amazon WorkSpaces Applications (formerly AppStream 2.0):** Available; renamed on 6 November 2025. Managed streaming of desktop applications, with G4, G5 and G6 GPU families. *Fit:* Works, but is aimed at many users and costs more effort to set up than needed for one person.

6.2 Instance Choice and Cost Breakdown (Weekly/Monthly)

I would choose g4dn.xlarge: 4 vCPUs, 16 GiB RAM and one NVIDIA T4 GPU with 16 GB of memory. Gazebo's GUI and its GPU lidar need a GPU, 16 GiB is double my laptop's RAM, and it is the smallest size in the g4dn family. Using us-east-1 Linux list prices (estimated around 0.526\$ per hour on-demand, spot around 0.25\$ to 0.30\$ per hour, and EBS gp3 at 0.08\$ per GB-month for 100 GB):

- **Weekly Cost (At 20 hours per week):**
 - On-Demand Instance (20 hours × \$0.526): \$10.52 per week
 - Spot Instance (20 hours × ~\$0.27): ~\$5.40 per week
 - Storage (EBS gp3 100 GB prorated weekly): ~\$1.85 per week
 - **Total Weekly Cost (On-Demand): ~\$12.37 per week**
- **Monthly Cost (At 86.7 hours per month):**
 - On-Demand Instance (86.7 hours × \$0.526): ~\$45.60 per month
 - Spot Instance (86.7 hours × ~\$0.27): ~\$23.40 per month
 - Storage (EBS gp3 100 GB fixed monthly): \$8.00 per month
 - **Total Monthly Cost (On-Demand): ~\$53.60 per month**

6.3 What Runs on the Drone and What Can Run in the Cloud

On the Drone (must work with no network)	In the Cloud (can tolerate delay)
Flight control and stabilisation loops, motor outputs, failsafes	Storing, sharing and merging maps; long-term log storage
Lidar driver and local obstacle avoidance	Offline map clean-up and global route planning
Real-time SLAM / localisation that feeds the position estimate	Simulation and regression testing (this project), dashboards and replay

The deciding factor is network delay. A cellular or Wi-Fi link can take tens to hundreds of milliseconds round trip and can drop out completely. At $2\sim m/s$ a drone moves 20 cm in 100 ms and a full metre in 500 ms, which is too far for collision avoidance. Even in my simulation the lidar arrives only every 200 ms (5 Hz), and I had to slow the drone to $0.4\sim m/s$ for avoidance at work. So anything inside the control loop or needed for safety stays on the drone, and the cloud handles work that is heavy, shared or not time critical.

References

- [1] GitHub, [aws-robotics/aws-robomaker-small-warehouse-world](#) (archived 21 Jul 2026).
- [12] AWS Robotics Blog, "Build headless robotic simulations with AWS Batch": [aws.amazon.com/blogs/robotics/build-headless-robotic-simulations-with-aws-batch/](#)
- [2] Amazon DCV product page: [aws.amazon.com/hpc/dcv](#) (read 7 Oct 2026).
- [3] Amazon WorkSpaces Applications (formerly AppStream 2.0): [aws.amazon.com/appstream2](#); rename notice 6 Nov 2025: [dev.classmethod.jp/articles/amazon-appstream2-rename-to-amazon-workspaces-applications/](#)